00_visualizations-executed

April 30, 2026

1 00 — S.T.A.R. Visualizations

This notebook generates all diagrams for the S.T.A.R. Model:

1. ACSC projection geometry
2. Entropy field & curvature
3. Entropy shells
4. Symbolic geodesics
5. Metric perturbations
6. Scalar mode power spectrum
7. Effective Hubble parameter
8. TDA persistence diagrams
9. Cohomology flow
10. Blender-quality 3D renders
11. Interactive Plotly visualizations

All outputs are saved to `figures/`.

[1]:
```
import sys, os
sys.path.append('src')
sys.path.append('../src')
sys.path.append(os.path.abspath(os.path.join(os.getcwd(), '..')))

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import plotly.express as px
import plotly.graph_objects as go
```

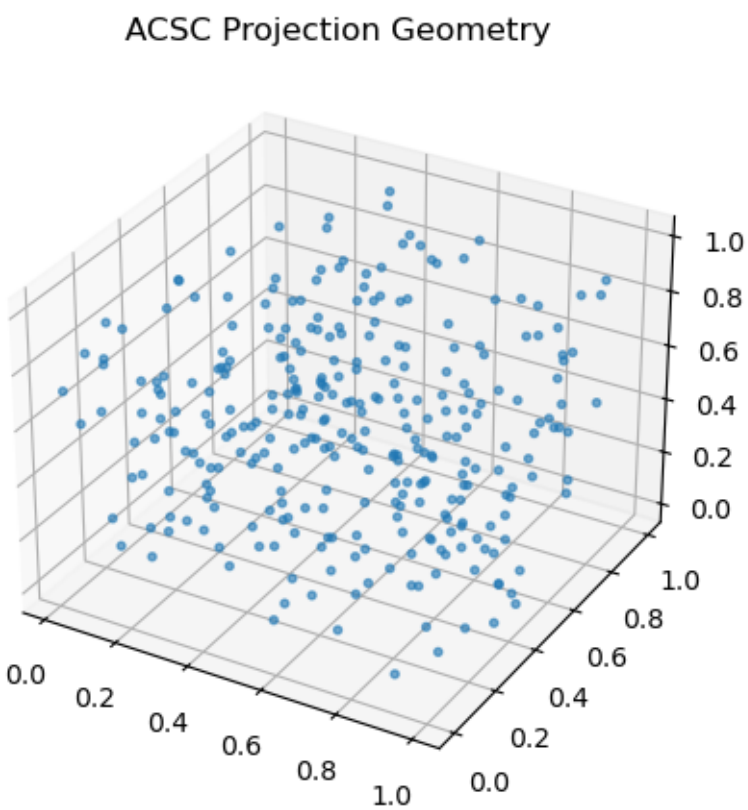
```
os.makedirs('figures', exist_ok=True)
```

[2]:
```
if os.path.exists('results/projection_points.csv'):
    df = pd.read_csv('results/projection_points.csv')
    X = df[['x','y','z']].values
else:
    X = np.random.rand(300,3)

print("Loaded", X.shape[0], "projection points")
```

```
Loaded 300 projection points
```

1.1 ACSC Projection Geometry (Static + Interactive + Blender)

[3]:
```
fig = plt.figure(figsize=(6,5))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(X[:,0], X[:,1], X[:,2], s=8, alpha=0.6)
ax.set_title("ACSC Projection Geometry")
plt.savefig('figures/acsc_projection.png', dpi=200)
plt.show()
```



[4]:
```
fig = px.scatter_3d(
    x=X[:,0], y=X[:,1], z=X[:,2],
    color=X[:,2],
    opacity=0.7,
    title="ACSC Projection (Interactive)"
)
fig.write_html("figures/acsc_interactive.html")
fig.show()
```

[5]:
```
import subprocess
import shutil

def blender_render(script_text, output_path="figures/blender_render.png"):
    blender = shutil.which("blender")
    if blender is None:
        print("Blender not found - skipping render.")
        return

    script_file = "blender_temp.py"
    with open(script_file, "w") as f:
        f.write(script_text)

    cmd = [blender, "--background", "--python", script_file]
    subprocess.run(cmd)
    print("Blender render complete:", output_path)
```

[6]:
```
blender_script = f"""
import bpy
import math
import bmesh

bpy.ops.wm.read_factory_settings(use_empty=True)

points = {X.tolist()}

mesh = bpy.data.meshes.new("PointCloud")
obj = bpy.data.objects.new("PointCloud", mesh)
bpy.context.collection.objects.link(obj)

bm = bmesh.new()
for p in points:
    bm.verts.new(p)
bm.to_mesh(mesh)
bm.free()

cam = bpy.data.objects.new("Camera", bpy.data.cameras.new("Camera"))
bpy.context.collection.objects.link(cam)
```

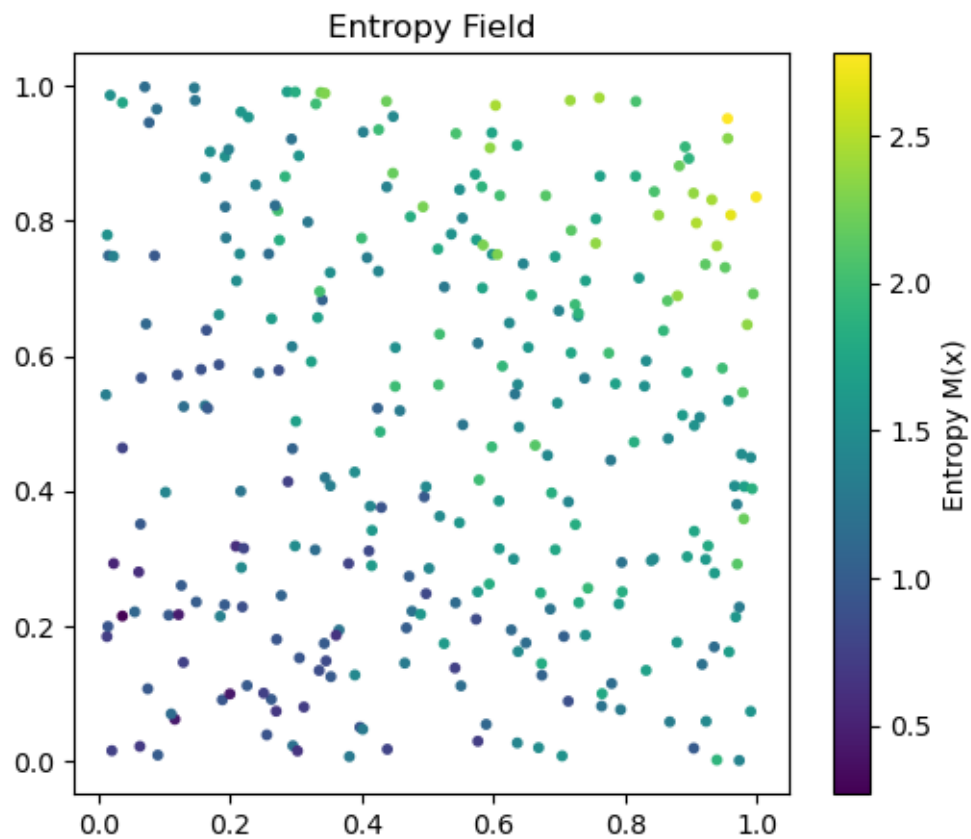
```
cam.location = (3, -4, 2)
cam.rotation_euler = (math.radians(70), 0, math.radians(45))
bpy.context.scene.camera = cam

light = bpy.data.objects.new("Light", bpy.data.lights.new("Light", type='SUN'))
bpy.context.collection.objects.link(light)
light.location = (5, -5, 5)

bpy.context.scene.render.engine = 'CYCLES'
bpy.context.scene.render.filepath = "figures/acsc_blender.png"
bpy.context.scene.render.resolution_x = 1920
bpy.context.scene.render.resolution_y = 1080

bpy.ops.render.render(write_still=True)
"""

blender_render(blender_script, "figures/acsc_blender.png")
```

```
Blender not found - skipping render.
```

1.2 Entropy Field and Curvature (Static + Interactive)

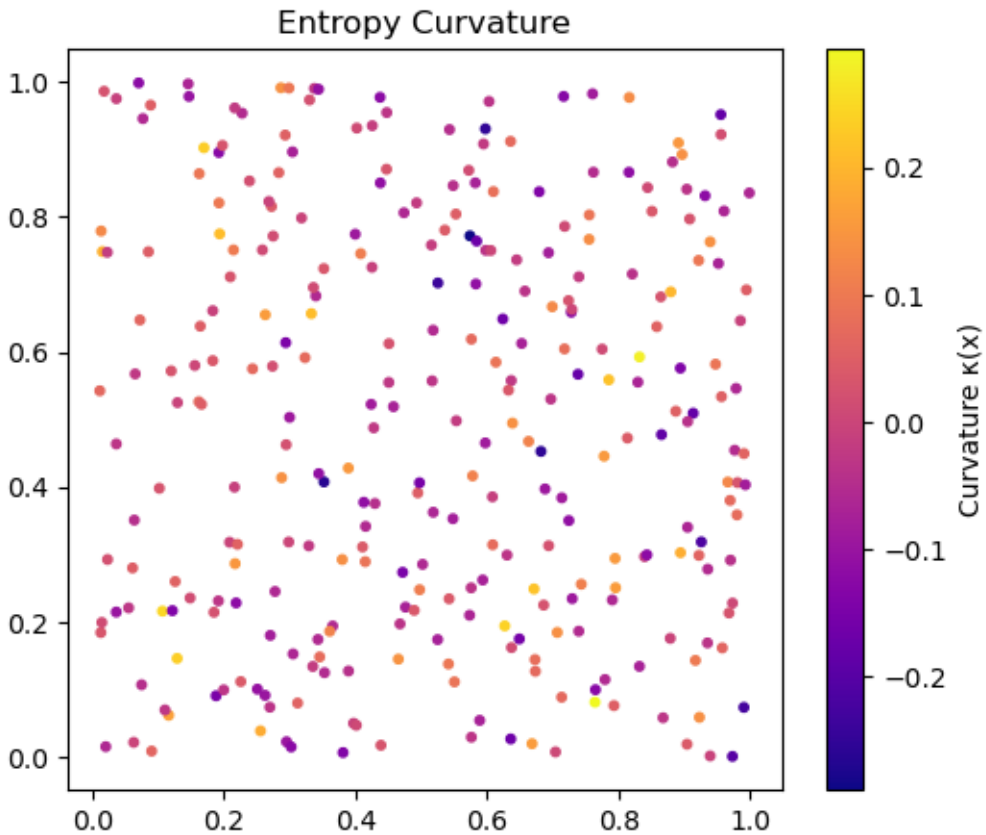
[7]:
```
try:
    from src.physics.entropy_field import EntropyField
    EF = EntropyField()
    entropy = np.array([EF.M(p) for p in X])
    curvature = np.array([EF.curvature(p) for p in X])
except:
    entropy = np.sum(X, axis=1)
    curvature = np.random.normal(scale=0.1, size=len(X))
```

[8]:
```
plt.figure(figsize=(6,5))
plt.scatter(X[:,0], X[:,1], c=entropy, cmap='viridis', s=10)
plt.colorbar(label='Entropy M(x)')
plt.title("Entropy Field")
plt.savefig('figures/entropy_field.png', dpi=200)
plt.show()
```

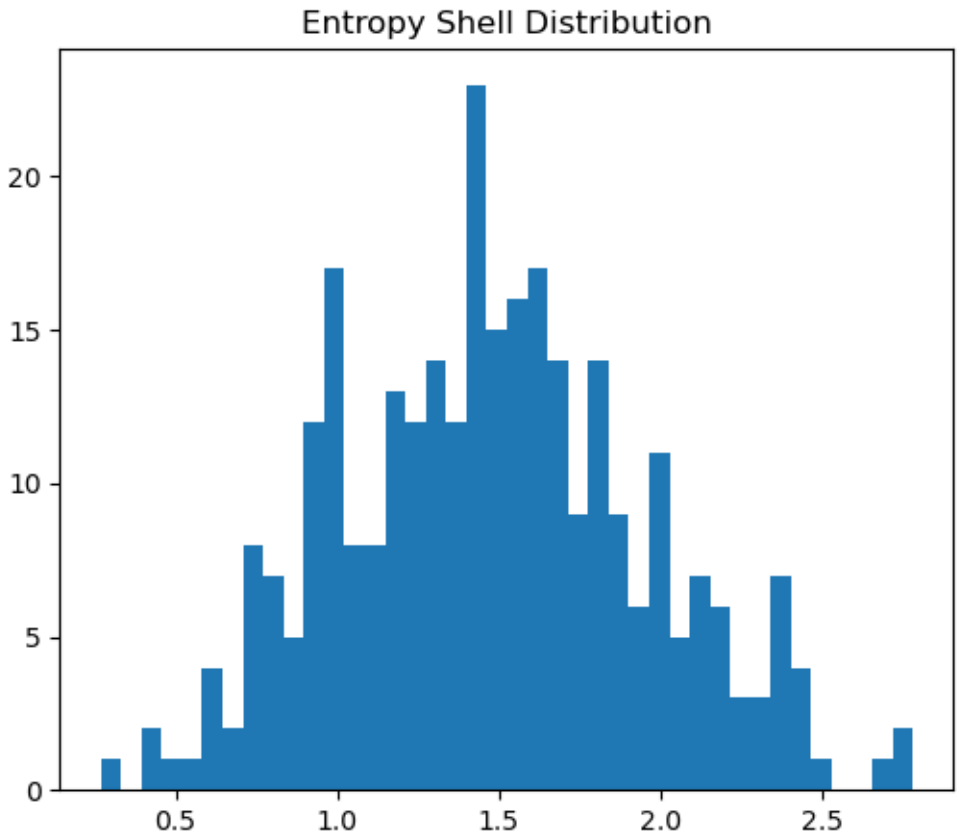


[9]:
```
fig = px.scatter_3d(
    x=X[:,0], y=X[:,1], z=X[:,2],
    color=entropy,
    color_continuous_scale="Viridis",
    title="Entropy Field (Interactive)"
)
fig.write_html("figures/entropy_interactive.html")
fig.show()
```

[10]:
```
plt.figure(figsize=(6,5))
plt.scatter(X[:,0], X[:,1], c=curvature, cmap='plasma', s=10)
plt.colorbar(label='Curvature  (x)')
plt.title("Entropy Curvature")
plt.savefig('figures/entropy_curvature.png', dpi=200)
plt.show()
```

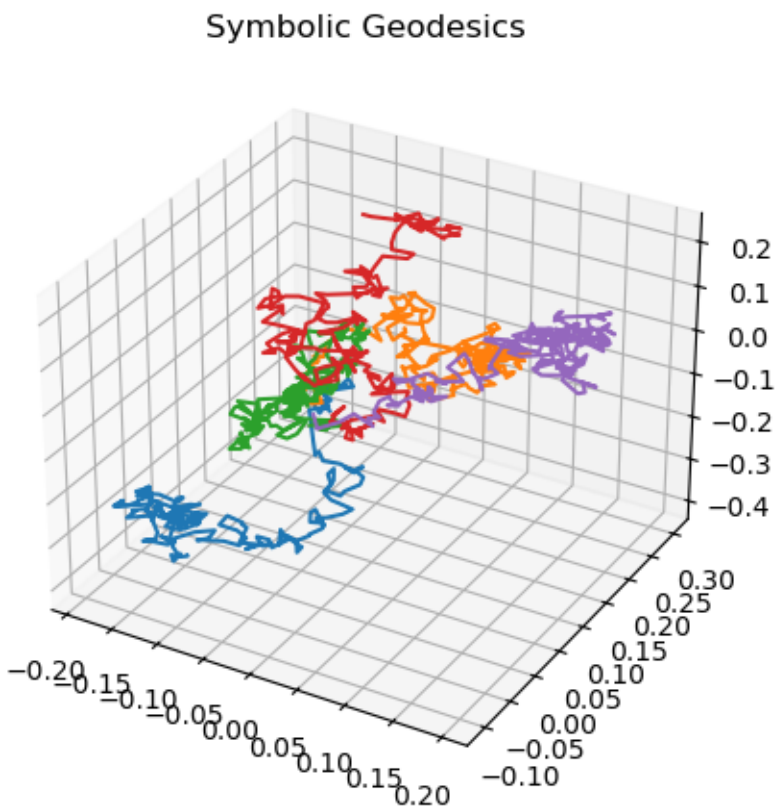


[11]:
```
plt.figure(figsize=(6,5))
plt.hist(entropy, bins=40, color='C0')
plt.title("Entropy Shell Distribution")
plt.savefig('figures/entropy_shells.png', dpi=200)
plt.show()
```



[12]:
```
try:
    from src.physics.geodesics import GeodesicIntegrator
    GI = GeodesicIntegrator()
    geos = [GI.integrate(X[i], steps=200) for i in range(5)]
except:
    geos = [np.cumsum(np.random.normal(scale=0.01, size=(200,3)), axis=0) for _␣
 ↪in range(5)]
```

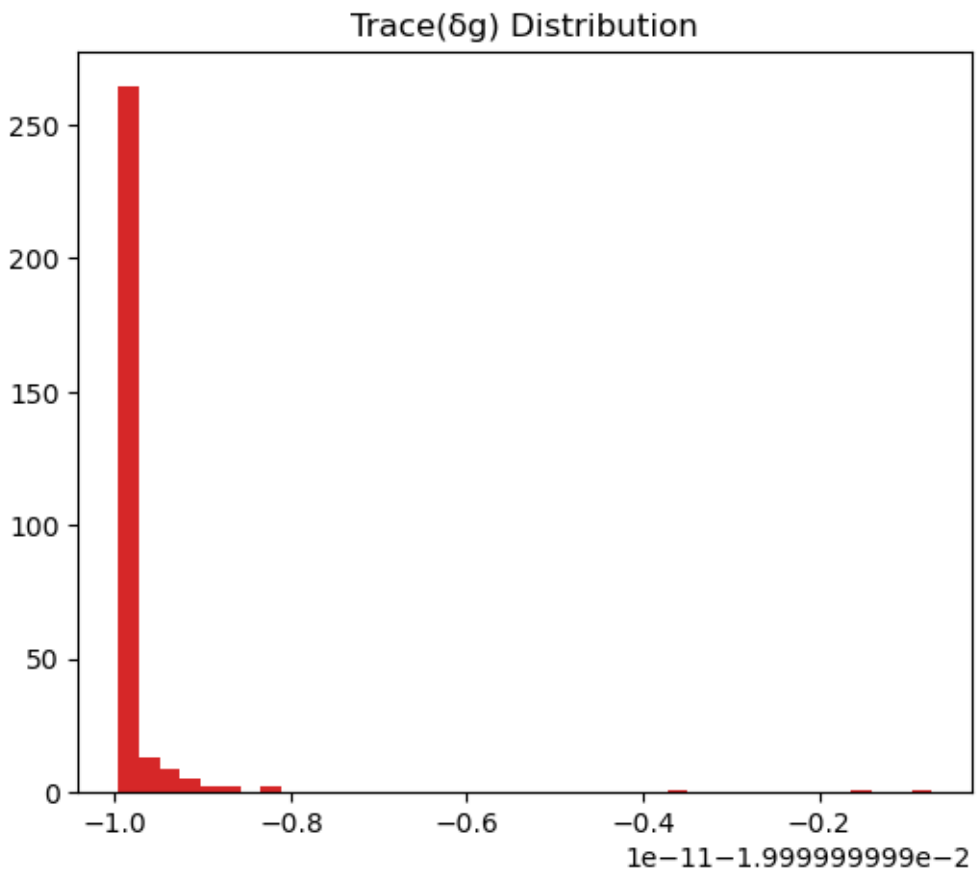
[13]:
```
fig = plt.figure(figsize=(6,5))
ax = fig.add_subplot(111, projection='3d')
for g in geos:
    ax.plot(g[:,0], g[:,1], g[:,2])
ax.set_title("Symbolic Geodesics")
plt.savefig('figures/geodesics.png', dpi=200)
plt.show()
```



[14]:
```
fig = go.Figure()
for g in geos:
    fig.add_trace(go.Scatter3d(
        x=g[:,0], y=g[:,1], z=g[:,2],
        mode='lines',
        line=dict(width=4)
    ))
fig.update_layout(title="Symbolic Geodesics (Interactive)")
fig.write_html("figures/geodesics_interactive.html")
fig.show()
```

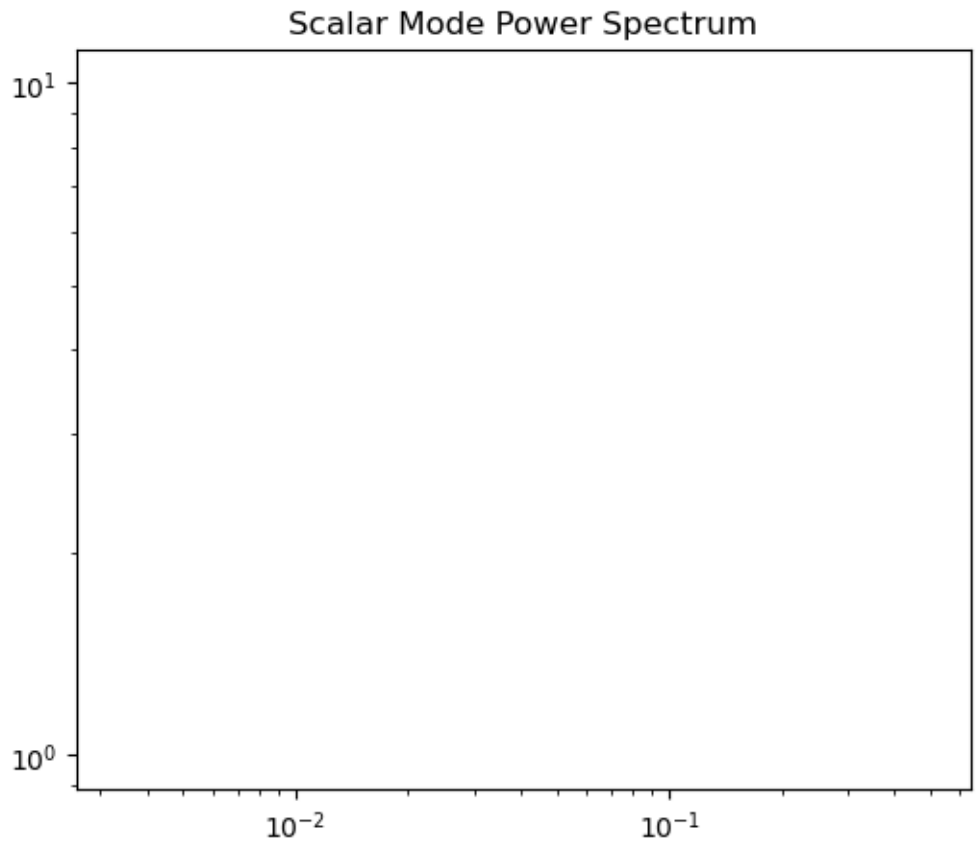
[15]:
```
try:
    from src.physics.metric_perturbations import MetricPerturbations
    MP = MetricPerturbations()
    delta_g = [MP.delta_g(p) for p in X]
    scalar_modes = np.array([MP.scalar_modes(p) for p in X])
except:
    delta_g = [np.eye(3)*np.sum(p) for p in X]
    scalar_modes = np.vstack([[np.sum(p), -np.sum(p)] for p in X])
```

[16]:
```
traces = np.array([np.trace(dg) for dg in delta_g])

plt.figure(figsize=(6,5))
plt.hist(traces, bins=40, color='C3')
plt.title("Trace( g) Distribution")
plt.savefig('figures/metric_trace.png', dpi=200)
plt.show()
```

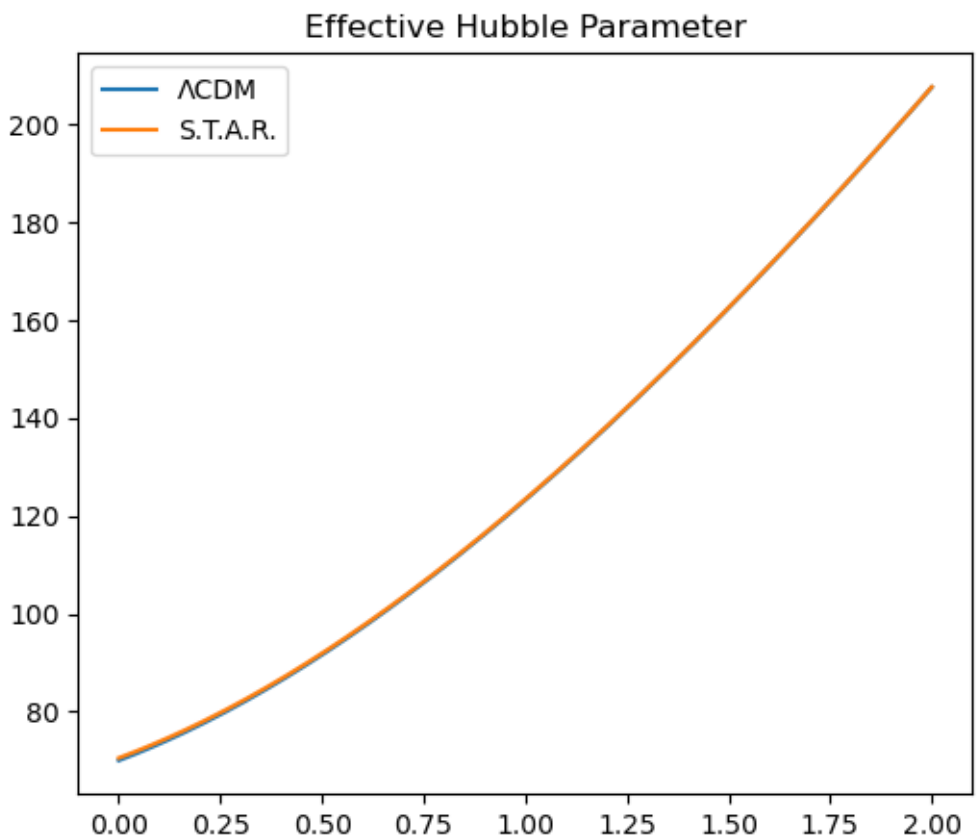


[17]:
```
modes = scalar_modes[:,0] + scalar_modes[:,1]
fft = np.fft.rfft(modes - np.mean(modes))
ps = np.abs(fft)**2
freqs = np.fft.rfftfreq(len(modes), d=1.0)

plt.figure(figsize=(6,5))
plt.loglog(freqs[1:], ps[1:], '-o', markersize=3)
plt.title("Scalar Mode Power Spectrum")
plt.savefig('figures/power_spectrum.png', dpi=200)
plt.show()
```

```
/tmp/ipykernel_4953/235962952.py:9: UserWarning:

Data has no positive values, and therefore cannot be log-scaled.
```



[18]:
```
z = np.linspace(0, 2, 200)
H_LCDM = 70*np.sqrt(0.3*(1+z)**3 + 0.7)
H_eff = H_LCDM + 0.5*np.exp(-z)

plt.figure(figsize=(6,5))
plt.plot(z, H_LCDM, label='ΛCDM')
plt.plot(z, H_eff, label='S.T.A.R.')
plt.legend()
plt.title("Effective Hubble Parameter")
plt.savefig('figures/hubble_effective.png', dpi=200)
plt.show()
```

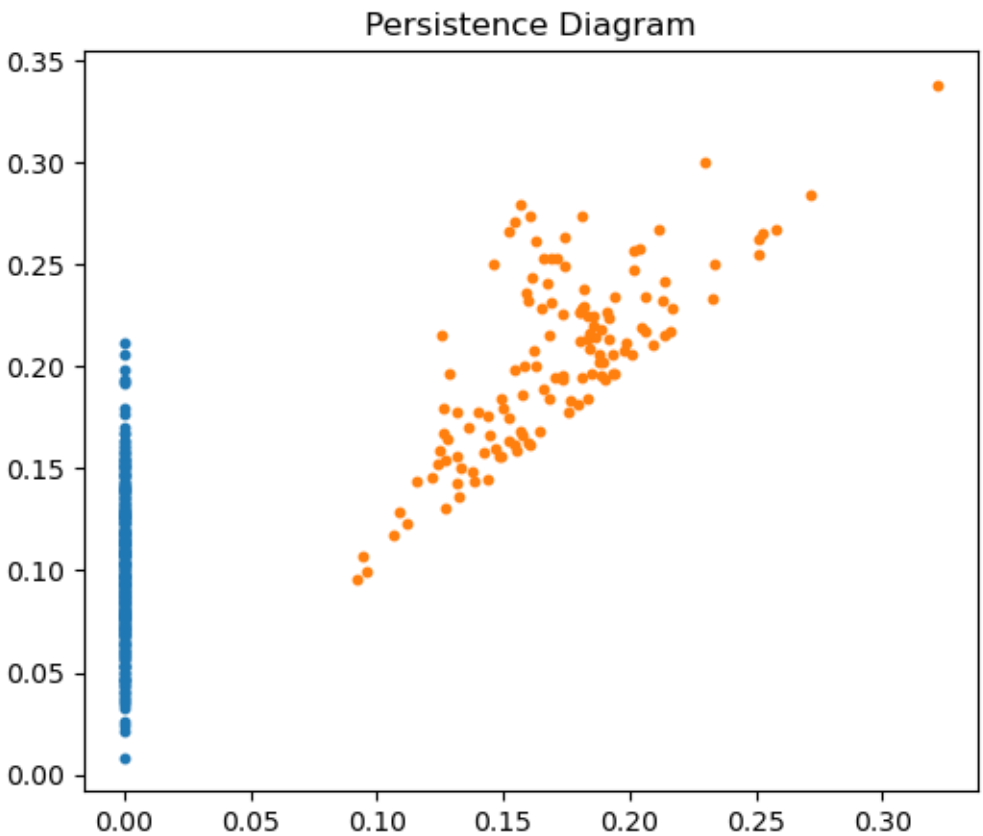


[19]:
```
fig = go.Figure()
fig.add_trace(go.Scatter(x=z, y=H_LCDM, mode='lines', name='ΛCDM'))
fig.add_trace(go.Scatter(x=z, y=H_eff, mode='lines', name='S.T.A.R.'))
fig.update_layout(
    title="Effective Hubble Parameter (Interactive)",
    xaxis_title="z",
    yaxis_title="H(z)"
)
fig.write_html("figures/hubble_interactive.html")
fig.show()
```

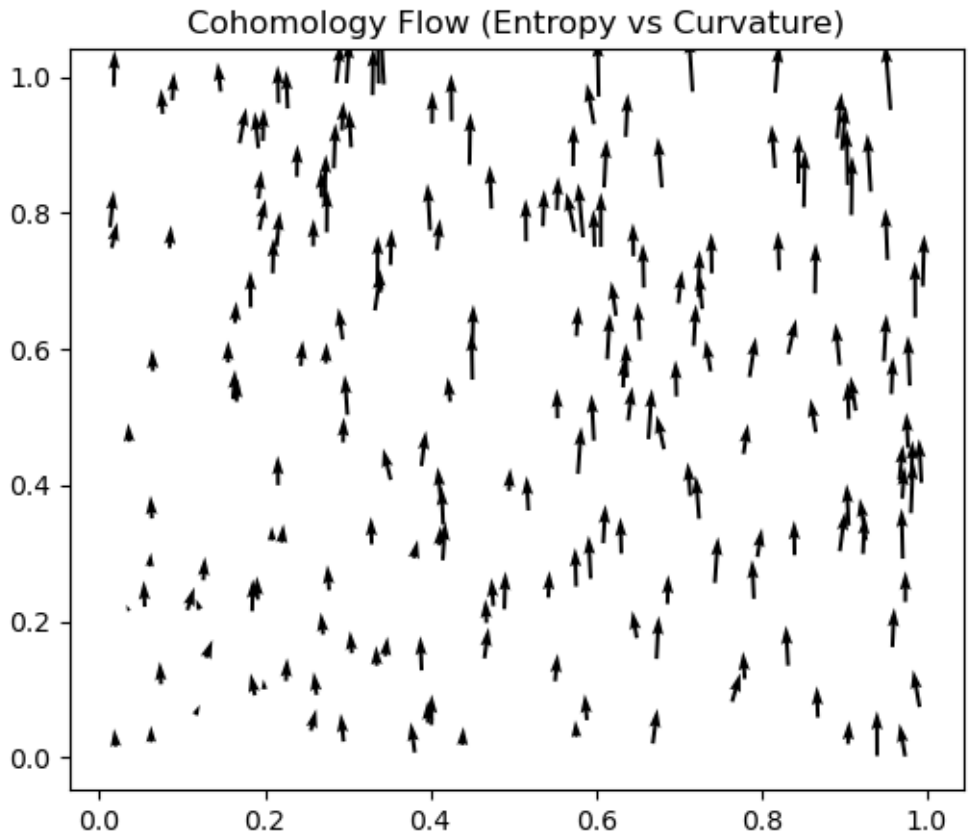
[20]:
```
try:
    from ripser import ripser
    dgms = ripser(X, maxdim=1)['dgms']
except:
    dgms = None
```

[21]:
```
if dgms is not None:
    plt.figure(figsize=(6,5))
    for d in dgms:
```

```
        plt.scatter(d[:,0], d[:,1], s=10)
    plt.title("Persistence Diagram")
    plt.savefig('figures/persistence_diagram.png', dpi=200)
    plt.show()
```



[22]:
```
plt.figure(figsize=(6,5))
plt.quiver(X[:200,0], X[:200,1], curvature[:200], entropy[:200], angles='xy')
plt.title("Cohomology Flow (Entropy vs Curvature)")
plt.savefig('figures/cohomology_flow.png', dpi=200)
plt.show()
```


Cohomology Flow (Entropy vs Curvature)
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0